

Библиотеки управления оборудованием

Содержание

1	Вводная часть	3
1.1	Общие принципы работы	3
1.2	Обработка ошибок	3
1.3	Общие методы	3
2	Модуль монохроматора	4
2.1	Создание объекта	4
2.2	Подключение и отключение	4
2.3	Управление длиной волны	4
2.4	Управление решётками	4
2.5	Управление щелями	5
2.6	Управление зеркалами	5
2.7	Управление фильтрами	6
2.8	Управление затвором	6
2.9	Получение информации	6
2.10	Константы модуля	7
3	Модуль лазера	7
3.1	Создание объекта	7
3.2	Подключение и отключение	7
3.3	Управление длиной волны	7
3.4	Управление двигателем	7
3.5	Управление затвором	8
3.6	Сброс устройства	8
3.7	Получение информации	8
3.8	Константы модуля	8
4	Модуль осциллографа	8
4.1	Создание объекта	8
4.2	Подключение и отключение	9
4.3	Управление каналами	9
4.4	Управление временной развёрткой	10
4.5	Управление триггером	10
4.6	Управление захватом	11
4.7	Захват данных	11
4.8	Измерения сигнала	12
4.8.1	Базовые измерения	12
4.8.2	Временные измерения	12
4.8.3	Специальные измерения	12
4.9	Курсоры	13

4.10	Математические функции	13
4.11	Сохранение данных	13
4.12	Сетевая информация	14
4.13	Управление ошибками	14
4.14	Получение информации	14
4.15	Константы модуля	14
4.15.1	Типы связи каналов	14
4.15.2	Режимы усреднения	14
4.15.3	Наклон триггера	14
4.15.4	Режимы триггера	15
4.15.5	Математические функции	15
4.15.6	Оконные функции	15
4.15.7	Режимы курсоров	15
4.15.8	Режимы временной развёртки	15
4.15.9	Ограничения полосы пропускания	15
4.16	Важно о таймаутах	16
5	Модуль энергометра	16
5.1	Создание объекта	16
5.2	Подключение и отключение	16
5.3	Измерения параметров	16
5.4	Управление шкалой	16
5.5	Настройка длины волны	17
5.6	Дополнительные настройки	17
5.7	Получение информации	18
5.8	Константы модуля	18
6	Практические примеры	18
6.1	Подключение всех устройств	18
6.2	Захват сигнала с осциллографа	19
6.3	Проведение измерения с монохроматором	19
6.4	Измерение параметров сигнала	19
6.5	Статистика сигнала	19
6.6	Сохранение результатов	20
6.7	Завершение работы	20

1 Вводная часть

Библиотеки предназначены для управления лабораторным оборудованием через язык Python. Каждая библиотека предоставляет удобный интерфейс для работы с конкретным типом устройства. Все библиотеки следуют единому стилю.

1.1 Общие принципы работы

- Все библиотеки используют единый стиль именования методов.
- Методы возвращают `True` при успешном выполнении и `False` при ошибке (для булевых операций).
- При ошибках методы возвращают `None`, `0.0`, или `[]` в зависимости от типа возвращаемого значения.
- Для получения подробной информации об ошибках используйте соответствующие методы получения ошибок.

1.2 Обработка ошибок

Все методы возвращают следующие значения при ошибках:

- Для методов, возвращающих `bool`: `False`.
- Для методов, возвращающих `float`: `0.0` или `None`.
- Для методов, возвращающих `str`: `.`
- Для методов, возвращающих `List`: `[]`.
- Для методов, возвращающих `Dict`: `{}`.

Для получения подробной информации об ошибках используйте:

- `get_next_error()` — возвращает следующую ошибку из очереди.
- `get_all_errors()` — возвращает все ошибки из очереди.
- `clear_errors()` — очищает очередь ошибок.
- `get_last_error()` — возвращает текст последней ошибки (для монохроматора).

1.3 Общие методы

Все библиотеки предоставляют следующие методы:

- `connect()` — устанавливает соединение с устройством.
- `disconnect()` — закрывает соединение.
- `is_connected()` — проверяет наличие соединения.
- `get_device_information()` — возвращает информацию об устройстве.
- `get_all_settings()` — возвращает все текущие настройки.

2 Модуль монохроматора

Управление монохроматором через комплект средств разработки.

2.1 Создание объекта

```
1 from chromator import Chromator
2
3 # Обычное создание
4 chromator = Chromator()
5
6 # С указанием пути к SDK
7 chromator = Chromator(sdk="C:/path/to/sdk")
```

2.2 Подключение и отключение

- `connect()` — подключается к монохроматору, возвращает `True` при успехе.
- `disconnect()` — отключается от монохроматора.
- `is_connected()` — проверяет наличие подключения, возвращает `True` при наличии соединения.
- `get_last_error()` — возвращает текст последней ошибки.

2.3 Управление длиной волны

- `set_wavelength(wavelength)` — устанавливает длину волны в нанометрах.
- `set_wavelength_async(wavelength)` — асинхронно устанавливает длину волны в нанометрах.
- `get_wavelength()` — возвращает текущую длину волны в нанометрах.
- `is_valid_wavelength(wavelength)` — проверяет возможность установки указанной длины волны.
- `wait_for_wavelength_stable(wavelength, tolerance, timeout)` — ожидает стабилизации длины волны с заданным допуском в нанометрах и таймаутом в секундах.

2.4 Управление решётками

- `get_grating_count()` — возвращает количество доступных решёток.
- `get_active_grating()` — возвращает индекс активной решётки.
- `set_active_grating(index)` — устанавливает активную решётку по индексу.
- `set_active_grating_async(index)` — асинхронно устанавливает активную решётку по индексу.

- `get_grating_parameters(index)` — возвращает параметры решётки: количество штрихов на миллиметр, минимальную длину волны, максимальную длину волны и угол блеска.
- `reset_grating()` — сбрасывает решётку в реперное положение.
- `reset_grating_async()` — асинхронно сбрасывает решётку в реперное положение.
- `reset_and_set_wavelength(wavelength)` — сбрасывает решётку и устанавливает указанную длину волны.
- `get_dispersion()` — возвращает дисперсию активной решётки.

2.5 Управление щелями

- `get_slit_count()` — возвращает количество щелей.
- `get_slit_name(index)` — возвращает название щели по её индексу.
- `set_slit_width(index, width, reset)` — устанавливает ширину щели в микрометрах с возможностью сброса перед установкой.
- `set_slit_width_async(index, width, reset)` — асинхронно устанавливает ширину щели в микрометрах с возможностью сброса перед установкой.
- `get_slit_width(index)` — возвращает текущую ширину щели в микрометрах.

2.6 Управление зеркалами

- `get_mirror_count()` — возвращает количество зеркал.
- `get_mirror_name(index)` — возвращает название зеркала по его индексу.
- `get_mirror_state_count(index)` — возвращает количество состояний зеркала по его индексу.
- `get_mirror_state_name(mirror, state)` — возвращает название состояния зеркала по индексам зеркала и состояния.
- `get_mirror_state(index)` — возвращает текущее состояние зеркала по его индексу.
- `set_mirror_state(mirror, state)` — устанавливает состояние зеркала по индексам зеркала и состояния.
- `set_mirror_state_async(mirror, state)` — асинхронно устанавливает состояние зеркала по индексам зеркала и состояния.

2.7 Управление фильтрами

- `get_filter_count()` — возвращает количество фильтров.
- `get_filter_name(index)` — возвращает название фильтра по его индексу.
- `get_filter_state_count(index)` — возвращает количество состояний фильтра по его индексу.
- `get_filter_state_name(filter, state)` — возвращает название состояния фильтра по индексам фильтра и состояния.
- `get_filter_bandwidth(filter, state)` — возвращает полосу пропускания фильтра по индексам фильтра и состояния.
- `get_filter_state(index)` — возвращает текущее состояние фильтра по его индексу.
- `set_filter_state(filter, state)` — устанавливает состояние фильтра по индексам фильтра и состояния.
- `set_filter_state_async(filter, state)` — асинхронно устанавливает состояние фильтра по индексам фильтра и состояния.

2.8 Управление затвором

- `get_shutter_count()` — возвращает количество затворов.
- `get_shutter_name(index)` — возвращает название затвора по его индексу.
- `shutter_open(index)` — открывает затвор по его индексу.
- `shutter_open_async(index)` — асинхронно открывает затвор по его индексу.
- `shutter_close(index)` — закрывает затвор по его индексу.
- `shutter_close_async(index)` — асинхронно закрывает затвор по его индексу.
- `get_shutter_state(index)` — возвращает состояние затвора по его индексу.

2.9 Получение информации

- `get_instrument_name()` — возвращает название прибора.
- `get_instrument_serial()` — возвращает серийный номер прибора.
- `get_device_information()` — возвращает словарь с информацией об устройстве.
- `get_all_settings()` — возвращает словарь со всеми текущими настройками.

2.10 Константы модуля

- `STATUS_READY = 1` — устройство готово к выполнению команд.
- `STATUS_BUSY = 2` — устройство выполняет команду.
- `STATUS_ERROR = -1` — произошла ошибка.
- `SHUTTER_CLOSED = 0` — затвор закрыт.
- `SHUTTER_OPENED = 1` — затвор открыт.
- `SHUTTER_MOVING = 2` — затвор находится в движении.

3 Модуль лазера

Управление источником лазерного излучения через последовательный порт.

3.1 Создание объекта

```
1 from laser_source import LaserSource
2
3 laser = LaserSource()
```

3.2 Подключение и отключение

- `connect(port, baudrate)` — подключается к лазеру через указанный порт и скорость, выполняет автоматический поиск если порт не указан.
- `disconnect()` — отключается от лазера.
- `is_connected()` — проверяет наличие подключения, возвращает `True` при наличии соединения.

3.3 Управление длиной волны

- `set_wavelength(wavelength)` — устанавливает длину волны в нанометрах.
- `get_wavelength()` — возвращает текущую длину волны в нанометрах.
- `wait_for_wavelength_stable(wavelength, tolerance, timeout)` — ожидает стабилизации длины волны с заданным допуском в нанометрах и таймаутом в секундах.

3.4 Управление двигателем

- `get_position(motor)` — возвращает текущее положение двигателя в шагах.
- `set_absolute_position(motor, position)` — устанавливает абсолютное положение двигателя в шагах.
- `set_relative_position(motor, steps)` — смещает двигатель на указанное число шагов.

- `get_speed(motor)` — возвращает текущую скорость двигателя в шагах в секунду.
- `set_speed(motor, speed)` — устанавливает скорость двигателя в шагах в секунду.
- `enable_motor(motor)` — включает питание двигателя.
- `disable_motor(motor)` — отключает питание двигателя.
- `get_status(motor)` — возвращает статус двигателя.
- `is_ready(motor)` — проверяет готовность двигателя, возвращает `True` если двигатель готов.

3.5 Управление затвором

- `set_shutter(number, open)` — управляет затвором, при значении `True` открывает, при `False` закрывает.
- `get_shutter(number)` — возвращает состояние затвора, `True` если открыт.

3.6 Сброс устройства

- `reset()` — выполняет полный сброс лазера.

3.7 Получение информации

- `get_model()` — возвращает модель лазера.
- `get_device_information()` — возвращает словарь с информацией об устройстве.
- `get_all_settings()` — возвращает словарь со всеми текущими настройками.

3.8 Константы модуля

- `STATUS_READY = 0` — двигатель готов к работе.
- `STATUS_BUSY = 1` — двигатель находится в движении.
- `STATUS_ERROR = -1` — произошла ошибка.

4 Модуль осциллографа

Управление цифровым осциллографом через интерфейс VISA.

4.1 Создание объекта

```
1 from oscilloscope import Oscilloscope
2
3 scope = Oscilloscope()
```


4.2 Подключение и отключение

- `connect(resource, timeout)` — подключается к осциллографу по указанному ресурсу, выполняет автоматический поиск если ресурс не указан, таймаут задаётся в миллисекундах.
- `disconnect()` — отключается от осциллографа.
- `is_connected()` — проверяет наличие подключения.
- `set_timeout(milliseconds)` — устанавливает таймаут операций в миллисекундах.
- `get_identification()` — возвращает строку идентификации осциллографа.

4.3 Управление каналами

- `set_channel_enabled(channel, enable)` — включает или отключает указанный канал.
- `is_channel_enabled(channel)` — проверяет включён ли указанный канал, возвращает `True` если включён.
- `set_channel_scale(channel, scale)` — устанавливает вертикальный масштаб указанного канала в вольтах на деление.
- `get_channel_scale(channel)` — возвращает вертикальный масштаб указанного канала в вольтах на деление.
- `set_channel_offset(channel, offset)` — устанавливает вертикальное смещение указанного канала в вольтах.
- `get_channel_offset(channel)` — возвращает вертикальное смещение указанного канала в вольтах.
- `set_channel_coupling(channel, coupling)` — устанавливает тип связи указанного канала.
- `get_channel_coupling(channel)` — возвращает тип связи указанного канала.
- `set_channel_impedance(channel, impedance)` — устанавливает входное сопротивление указанного канала в омах (50.0 или 1e6).
- `get_channel_impedance(channel)` — возвращает входное сопротивление указанного канала в омах.
- `set_channel_probe_attenuation(channel, attenuation)` — устанавливает коэффициент ослабления зонда.
- `get_channel_probe_attenuation(channel)` — возвращает коэффициент ослабления зонда.
- `set_channel_inverted(channel, invert)` — инвертирует сигнал на указанном канале.
- `is_channel_inverted(channel)` — проверяет инвертирован ли сигнал на указанном канале.

- `set_channel_bandwidth_limit(channel, limit)` — ограничивает полосу пропускания указанного канала.
- `get_channel_bandwidth_limit(channel)` — возвращает ограничение полосы пропускания указанного канала.
- `set_channel_label(channel, label)` — устанавливает метку канала (до 10 символов).
- `get_channel_label(channel)` — возвращает метку канала.
- `auto_scale()` — автоматически настраивает масштаб всех каналов.

4.4 Управление временной развёрткой

- `set_timebase_scale(scale)` — устанавливает горизонтальный масштаб в секундах на деление.
- `get_timebase_scale()` — возвращает горизонтальный масштаб в секундах на деление.
- `set_timebase_delay(delay)` — устанавливает задержку развёртки в секундах.
- `get_timebase_delay()` — возвращает задержку развёртки в секундах.
- `set_timebase_reference(reference)` — устанавливает опорную точку развёртки (LEFT, CENTER, RIGHT).
- `get_timebase_reference()` — возвращает опорную точку развёртки.
- `set_timebase_mode(mode)` — устанавливает режим развёртки.
- `get_timebase_mode()` — возвращает режим развёртки.

4.5 Управление триггером

- `set_trigger_source(source)` — устанавливает источник триггера.
- `get_trigger_source()` — возвращает источник триггера.
- `set_trigger_level(level)` — устанавливает уровень триггера в вольтах.
- `get_trigger_level()` — возвращает уровень триггера в вольтах.
- `set_trigger_slope(slope)` — устанавливает наклон триггера.
- `get_trigger_slope()` — возвращает наклон триггера.
- `set_trigger_mode(mode)` — устанавливает режим триггера.
- `get_trigger_mode()` — возвращает режим триггера.
- `set_trigger_coupling(coupling)` — устанавливает тип связи триггера.
- `get_trigger_coupling()` — возвращает тип связи триггера.
- `set_trigger_holdoff(holdoff)` — устанавливает задержку триггера в секундах.

- `get_trigger_holdoff()` — возвращает задержку триггера в секундах.
- `force_trigger()` — принудительно запускает триггер.
- `get_trigger_status()` — возвращает статус триггера.

4.6 Управление захватом

- `run_acquisition()` — запускает непрерывный захват сигнала.
- `stop_acquisition()` — останавливает захват сигнала.
- `single_acquisition()` — выполняет однократный захват сигнала.
- `is_acquisition_complete()` — проверяет завершение захвата, возвращает `True` если захват завершён.
- `set_acquisition_type(type)` — устанавливает режим усреднения.
- `get_acquisition_type()` — возвращает режим усреднения.
- `set_average_count(count)` — устанавливает количество кадров для усреднения.
- `get_average_count()` — возвращает количество кадров для усреднения.
- `get_acquisition_status()` — возвращает словарь со статусом захвата.
- `wait_for_ready(timeout)` — ожидает готовности осциллографа к выполнению команд.
- `reset()` — выполняет сброс осциллографа к настройкам по умолчанию.
- `self_test()` — выполняет встроенный тест осциллографа.

4.7 Захват данных

- `capture_waveform(channel, points)` — захватывает форму сигнала с указанного канала с заданным количеством точек, возвращает кортеж (`time_values`, `voltage_values`).
- `acquire_averaged_waveform(channel, average, points, timeout)` — захватывает усреднённый сигнал с указанного канала с заданным количеством точек и числом кадров усреднения.
- `acquire_averaged_waveform_retry(channel, average, points, retries, timeout)` — захватывает усреднённый сигнал с указанным числом попыток в случае ошибки.

Формат возвращаемых данных Методы `capture_waveform` и `acquire_averaged_waveform` возвращают кортеж (`time_values`, `voltage_values`):

- `time_values` — список значений времени в секундах (`List[float]`).
- `voltage_values` — список значений напряжения в вольтах (`List[float]`).

Длины обоих списков равны указанному количеству точек.

4.8 Измерения сигнала

4.8.1 Базовые измерения

- `measure_voltage_peak_to_peak(channel)` — измеряет напряжение пик-пик на указанном канале в вольтах.
- `measure_voltage_maximum(channel)` — измеряет максимальное напряжение на указанном канале в вольтах.
- `measure_voltage_minimum(channel)` — измеряет минимальное напряжение на указанном канале в вольтах.
- `measure_voltage_rms(channel)` — измеряет среднеквадратичное напряжение на указанном канале в вольтах.
- `measure_mean_voltage(channel)` — измеряет среднее напряжение на указанном канале в вольтах.
- `measure_ac_rms(channel)` — измеряет среднеквадратичное значение переменного тока на указанном канале в вольтах.

4.8.2 Временные измерения

- `measure_frequency(channel)` — измеряет частоту сигнала на указанном канале в герцах.
- `measure_period(channel)` — измеряет период сигнала на указанном канале в секундах.
- `measure_rise_time(channel)` — измеряет время нарастания сигнала на указанном канале в секундах.
- `measure_fall_time(channel)` — измеряет время спада сигнала на указанном канале в секундах.
- `measure_positive_width(channel)` — измеряет длительность положительного импульса на указанном канале в секундах.
- `measure_negative_width(channel)` — измеряет длительность отрицательного импульса на указанном канале в секундах.
- `measure_duty_cycle(channel)` — измеряет скважность сигнала на указанном канале в процентах.

4.8.3 Специальные измерения

- `measure_overshoot(channel)` — измеряет выброс сигнала на указанном канале в процентах.
- `measure_preshoot(channel)` — измеряет предвыброс сигнала на указанном канале в процентах.
- `measure_delay(source1, source2)` — измеряет задержку между двумя указанными источниками в секундах.

- `measure_phase(source1, source2)` — измеряет фазовый сдвиг между двумя указанными источниками в градусах.
- `measure_parameter(parameter, channel)` — универсальный метод измерения любого параметра.
- `compute_waveform_statistics(values)` — вычисляет статистику сигнала: максимум, минимум, размах, среднее, среднеквадратичное значение и стандартное отклонение.

4.9 Курсоры

- `set_cursor_mode(mode)` — устанавливает режим курсоров: OFF, X, Y, XY.
- `get_cursor_mode()` — возвращает текущий режим курсоров.
- `set_cursor_position(cursor, position)` — устанавливает положение курсора (X1P, X2P, Y1P, Y2P).
- `get_cursor_position(cursor)` — возвращает положение курсора.
- `get_cursor_deltas()` — возвращает словарь с разностями между курсорами: delta_x, delta_y, inverse_delta_x.

4.10 Математические функции

- `set_math_function(function)` — устанавливает математическую функцию: ADD, SUBT, MULT, DIV, FFT.
- `get_math_function()` — возвращает текущую математическую функцию.
- `set_math_sources(source1, source2)` — устанавливает источники для математической функции.
- `set_math_scale(scale)` — устанавливает масштаб математической функции.
- `get_math_scale()` — возвращает масштаб математической функции.
- `set_math_offset(offset)` — устанавливает смещение математической функции.
- `get_math_offset()` — возвращает смещение математической функции.
- `set_math_fft_window(window)` — устанавливает оконную функцию для БПФ: RECT, HAMM, HANN, BLAS.

4.11 Сохранение данных

- `save_screenshot(filename)` — сохраняет скриншот экрана осциллографа в указанный файл.
- `save_setup(location)` — сохраняет текущие настройки в ячейку памяти по указанному номеру.
- `recall_setup(location)` — восстанавливает настройки из ячейки памяти по указанному номеру.

4.12 Сетевая информация

- `get_ip_address()` — возвращает IP-адрес осциллографа.
- `get_mac_address()` — возвращает MAC-адрес осциллографа.

4.13 Управление ошибками

- `clear_errors()` — очищает очередь ошибок.
- `get_next_error()` — возвращает следующую ошибку из очереди.
- `get_all_errors()` — возвращает все ошибки из очереди.

4.14 Получение информации

- `get_identification()` — возвращает строку идентификации осциллографа.
- `get_device_information()` — возвращает словарь с информацией об устройстве.
- `get_all_settings()` — возвращает словарь со всеми текущими настройками.
- `get_acquisition_status()` — возвращает словарь со статусом захвата.

4.15 Константы модуля

4.15.1 Типы связи каналов

- `COUPLING_AC` = "AC" — связь по переменному току.
- `COUPLING_DC` = "DC" — связь по постоянному току.
- `COUPLING_GND` = "GND" — заземление входа.

4.15.2 Режимы усреднения

- `ACQ_NORMAL` = "NORM" — обычный режим.
- `ACQ_AVERAGE` = "AVER" — режим усреднения.
- `ACQ_PEAK` = "PEAK" — режим пикового детектора.
- `ACQ_HRES` = "HRES" — режим высокого разрешения.

4.15.3 Наклон триггера

- `SLOPE_POS` = "POS" — триггер по переднему фронту.
- `SLOPE_NEG` = "NEG" — триггер по заднему фронту.
- `SLOPE_EITH` = "EITH" — триггер по любому фронту.

4.15.4 Режимы триггера

- TRIG_AUTO = "AUTO" — автоматический режим.
- TRIG_NORM = "NORM" — нормальный режим.
- TRIG_SING = "SING" — однократный режим.

4.15.5 Математические функции

- MATH_ADD = "ADD" — сложение.
- MATH_SUBT = "SUBT" — вычитание.
- MATH_MULT = "MULT" — умножение.
- MATH_DIV = "DIV" — деление
- MATH_FFT = "FFT" — быстрое преобразование Фурье.

4.15.6 Оконные функции

- FFT_RECT = "RECT" — прямоугольное окно.
- FFT_HAMM = "HAMM" — окно Хэмминга.
- FFT_HANN = "HANN" — окно Ханна.
- FFT_BLAC = "BLAC" — окно Блэкмана.

4.15.7 Режимы курсоров

- CURSOR_OFF = "OFF" — курсоры выключены.
- CURSOR_X = "X" — курсоры по оси X.
- CURSOR_Y = "Y" — курсоры по оси Y.
- CURSOR_XY = "XY" — курсоры по осям X и Y.

4.15.8 Режимы временной развёртки

- TIMEBASE_MAIN = "MAIN" — основной режим.
- TIMEBASE_WINDOW = "WIND" — режим окна.
- TIMEBASE_XY = "XY" — режим XY.
- TIMEBASE_ROLL = "ROLL" — режим прокрутки.

4.15.9 Ограничения полосы пропускания

- BW_FULL = "FULL" — полная полоса пропускания.
- BW_20MHZ = "20MHZ" — ограничение 20 МГц.
- BW_100MHZ = "100MHZ" — ограничение 100 МГц.
- BW_200MHZ = "200MHZ" — ограничение 200 МГц.

4.16 Важно о таймаутах

- Таймаут по умолчанию составляет 15000 миллисекунд (15 секунд).
- Для длительных операций (усреднение с большим числом кадров) таймаут автоматически увеличивается до 30000 мс.
- При необходимости используйте `set_timeout(milliseconds)` для изменения таймаута.
- Операция `acquire_averaged_waveform` имеет собственный таймаут `timeout_seconds` (по умолчанию 120 секунд).

5 Модуль энергометра

Управление измерителем мощности и энергии лазерного излучения.

5.1 Создание объекта

```
1 from energymeter import Energymeter
2
3 meter = Energymeter()
```

5.2 Подключение и отключение

- `connect(port, baudrate)` — подключается к энергометру через указанный порт и скорость, выполняет автоматический поиск если порт не указан.
- `disconnect()` — отключается от энергометра.
- `is_connected()` — проверяет наличие подключения, возвращает `True` при наличии соединения.

5.3 Измерения параметров

- `get_power()` — возвращает текущую мощность в ваттах.
- `get_energy()` — возвращает текущую энергию в джоулях.
- `get_average_power(count, delay)` — возвращает усреднённую мощность за указанное число измерений с заданной задержкой между ними.
- `get_average_energy(count, delay)` — возвращает усреднённую энергию за указанное число измерений с заданной задержкой между ними.

5.4 Управление шкалой

- `set_scale(index)` — устанавливает шкалу по индексу от 0 до 41.
- `set_scale_up()` — увеличивает шкалу на одну позицию.
- `set_scale_down()` — уменьшает шкалу на одну позицию.

- `get_current_scale_index()` — возвращает текущий индекс шкалы.
- `set_autoscale(enable)` — включает автоматический выбор шкалы при значении `True` и отключает при `False`.
- `get_autoscale()` — проверяет включена ли автоматическая шкала, возвращает `True` если включена.

5.5 Настройка длины волны

- `set_wavelength_nanometers(wavelength)` — устанавливает длину волны в нанометрах.
- `set_wavelength_microns(wavelength)` — устанавливает длину волны в микрометрах.
- `get_wavelength()` — возвращает текущую длину волны в нанометрах.

5.6 Дополнительные настройки

- `set_trigger_level(level)` — устанавливает уровень триггера в процентах от нуля целых одной десятой до девяноста девяти целых девяти десятых.
- `get_trigger_level()` — возвращает уровень триггера в процентах.
- `set_noise_suppression(size)` — устанавливает подавление шума, принимает значения от нуля до девятисот девяноста девяти.
- `set_zero_offset()` — выполняет обнуление показаний.
- `clear_zero_offset()` — очищает обнуление показаний.
- `set_attenuator(enable)` — включает аттенюатор при значении `True` и отключает при `False`.
- `get_attenuator()` — проверяет состояние аттенюатора, возвращает `True` если включён.
- `set_user_multiplier(multiplier)` — устанавливает пользовательский множитель.
- `get_user_multiplier()` — возвращает пользовательский множитель.
- `set_user_offset(offset)` — устанавливает пользовательское смещение.
- `get_user_offset()` — возвращает пользовательское смещение.
- `set_baud_rate(index)` — устанавливает скорость порта по индексу от нуля до четырёх.

5.7 Получение информации

- `get_version()` — возвращает версию прошивки устройства.
- `get_status()` — возвращает статус устройства.
- `get_extended_status()` — возвращает расширенный статус устройства.
- `get_device_information()` — возвращает словарь с информацией об устройстве.
- `get_all_settings()` — возвращает словарь со всеми текущими настройками.

5.8 Константы модуля

- `MODE_POWER = 0` — режим измерения мощности.
- `MODE_ENERGY = 1` — режим измерения энергии.
- `BAUD_9600 = 0` — скорость порта 9600 бод.
- `BAUD_19200 = 1` — скорость порта 19200 бод.
- `BAUD_38400 = 2` — скорость порта 38400 бод.
- `BAUD_57600 = 3` — скорость порта 57600 бод.
- `BAUD_115200 = 4` — скорость порта 115200 бод.

6 Практические примеры

6.1 Подключение всех устройств

```
1 from chromator import Chromator
2 from laser_source import LaserSource
3 from oscilloscope import Oscilloscope
4 from energymeter import Energymeter
5
6 # Создание объектов
7 chromator = Chromator()
8 laser = LaserSource()
9 scope = Oscilloscope()
10 meter = Energymeter()
11
12 # Подключение
13 chromator.connect()
14 laser.connect()
15 scope.connect()
16 meter.connect()
```

6.2 Захват сигнала с осциллографа

```
1 # Настройка осциллографа
2 scope.set_channel_enabled(1, True)
3 scope.set_channel_scale(1, 1.0)
4 scope.set_timebase_scale(0.001)
5 scope.run_acquisition()
6
7 # Обычный захват
8 time_values, volt_values = scope.capture_waveform(1, 2000)
9
10 # Захват с усреднением (64 кадра)
11 time_average, volt_average = scope.acquire_averaged_waveform(
12     channel=1,
13     average_count=64,
14     points_count=2000
15 )
```

6.3 Проведение измерения с монохроматором

```
1 # Настройка длины волны
2 chromator.set_wavelength(500.0)
3 laser.set_wavelength(500.0)
4
5 # Ожидание стабилизации
6 chromator.wait_for_wavelength_stable(500.0, 0.1, 10.0)
7 laser.wait_for_wavelength_stable(500.0, 0.1, 10.0)
8
9 # Измерение мощности
10 power = meter.get_power()
11 print(f"Мощность: {power:.6e} Вт")
```

6.4 Измерение параметров сигнала

```
1 # Измерение параметров сигнала на осциллографе
2 frequency = scope.measure_frequency(1)
3 voltage = scope.measure_voltage_peak_to_peak(1)
4 rise = scope.measure_rise_time(1)
5
6 print(f"Измеренная частота: {frequency:.3f} Гц")
7 print(f"Напряжение пик-пик: {voltage:.3f} В")
8 print(f"Время нарастания: {rise:.6f} с")
```

6.5 Статистика сигнала

```
1 # Вычисление статистических параметров захваченного сигнала
2 statistics = scope.compute_waveform_statistics(volt_values)
3
4 print(f"Максимум: {statistics['maximum_voltage']:.3f} В")
5 print(f"Минимум: {statistics['minimum_voltage']:.3f} В")
```

```

6 print(f"Пазмах: {statistics['peak_to_peak_voltage']:.3f} B")
7 print(f"Среднее: {statistics['mean_voltage']:.3f} B")
8 print(f"Эффективное значение: {statistics['rms_voltage']:.3f} B")
9 print(f"Стандартное отклонение: {statistics['standard_deviation']:.3f} B")

```

6.6 Сохранение результатов

```

1 # Сохранение данных
2 with open("data.csv", "w") as new_file:
3     new_file.write("time_seconds,voltage_volts\n")
4
5     for time_value, volt_value in zip(time_values, volt_values):
6         new_file.write(f"{time_value:.8e},{volt_value:.6f}\n")
7
8 # Сохранение скриншота
9 scope.save_screenshot("scope_screenshot.png")

```

6.7 Завершение работы

```

1 # Отключение устройств
2 chromator.disconnect()
3 laser.disconnect()
4 scope.disconnect()
5 meter.disconnect()

```